

EFFICIENT FILE ALLOCATION AND DISK-SCHEDULING STRATEGIES UNDERLIE THE STORAGE BACKENDS OF CLOUD MEDIA PLATFORMS, STREAMING SERVICES, AND CONTENT-DELIVERY SYSTEMS USED ACROSS INDUSTRY.

COMMON COURSE ASSIGNMENT

CSA04 – OPERATING SYSTEMS | Covering CO4 (Unit IV)

A. Assignment Information

Name of the student	J. Renuka Devi
Register Number	192573007
Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA0402– Operating Systems
Academic Year / Batch	2026 – 2027
Faculty Name	Dr.S.John Justin Thangaraj
Assignment Title	Design and Analysis of File Allocation Strategies and Disk Scheduling Algorithms for a Cloud-Based Digital Media Platform
Date of Issue	21.08.2026
Date of Submission	24.08.2026
Maximum Marks	100
Course Outcome(s) – CO	CO4 - Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Efficient file allocation and disk-scheduling strategies underlie the storage backends of cloud media platforms, streaming services, and content-delivery systems used across industry.

1. REAL-TIME SCENARIO

Modern universities depend on computer systems to provide continuous services to students, faculty and administrative staff. Learning platforms, online examination systems, digital classrooms, library services, research applications, file storage and backup systems may all operate simultaneously.

The workload becomes especially demanding during **online examination periods**, when a large number of students access the system at the same time. Multiple processes compete for CPU time, shared examination data must be accessed safely, and limited memory, storage and I/O resources must be managed efficiently.

For example, while students are logging into an online examination system, background processes may simultaneously perform database operations, generate reports, store submitted answers, execute research tasks and perform backups.

Such a workload creates several Operating System challenges:

- CPU processes may experience long waiting times.
- Multiple processes may attempt to access shared data simultaneously.
- Resource allocation may lead to an unsafe state or deadlock.
- Heavy memory usage may increase page faults.
- Different types of university files require different allocation strategies.
- Large numbers of disk requests may increase disk-head movement.

To study these challenges, **UniCore** is developed as an integrated Operating System resource-management simulator. The system models a smart university workload and applies different OS algorithms to manage CPU scheduling, synchronization, deadlock, memory, files and disk requests.

The objective is not simply to implement individual algorithms, but to **compare their behaviour and determine which strategies are more suitable for the selected university workload**.

2. CODING / DESIGN CHALLENGE

The main coding and design challenge is to create a single Operating System simulation that combines several important OS concepts while keeping each component independently testable.

The system must represent university activities as processes and resources and evaluate how different algorithms perform under normal and peak workloads.

The following OS techniques are implemented:

OS Area	Algorithms / Technique
CPU Scheduling	FCFS, Round Robin
Synchronization	Readers–Writers
Deadlock Avoidance	Banker’s Algorithm
Page Replacement	FIFO, LRU, Optimal
File Allocation	Contiguous, Linked, Indexed
Disk Scheduling	FCFS, SSTF, SCAN

The major design challenge is to ensure that each algorithm produces measurable results that can later be compared.

For CPU scheduling, the system calculates waiting time, turnaround time and response time. For memory management, page faults are counted. For disk scheduling, total head movement is calculated. For deadlock management, the system determines whether a state is safe or unsafe.

The final system uses these results to make an **engineering decision based on the workload rather than simply selecting an algorithm theoretically.**

3. STUDENT WORK

3.1 Problem Formulation

The UniCore workload represents a smart university environment with multiple concurrent services.

The following processes are considered:

Process	University Service	Arrival Time	Burst Time
P1	Examination Login	0	5
P2	Examination Database	1	8
P3	Digital Classroom	2	4
P4	Library Search	3	3
P5	Research Computation	4	10
P6	Backup Service	5	7
P7	IoT Monitoring	6	4
P8	Report Generation	7	6

The peak examination workload is considered as the primary stress scenario.

Parameter	Value
Concurrent Users	800
CPU Processes	8
Resource Types	4
Physical RAM	16 GB
Page Size	4 KB
Disk Cylinders	0–199
Round Robin Time Quantum	3
Memory Frames	3 and 4

3.2 Assumptions

The following assumptions are made for the simulation:

1. Each university service is represented as a process.
2. CPU scheduling is performed on a single simulated CPU.
3. Round Robin uses a time quantum of 3 units.
4. The memory system uses paging.
5. FIFO, LRU and Optimal are tested using the same reference string.
6. Three and four memory frames are considered for comparison.
7. Four resource types are used for Banker's Algorithm.

8. The disk contains cylinders numbered from 0 to 199.
9. Disk requests are generated by university services.
10. The simulation uses fixed input data so that different algorithms can be compared fairly.

3.3 CPU Scheduling

FCFS

First Come First Serve schedules processes according to their arrival order.

PSEUDOCODE:

BEGIN

Input number of processes

For each process:

Input Process ID

Input Arrival Time

Input Burst Time

Sort all processes according to Arrival Time

Set CurrentTime = 0

For each process in sorted order:

If CurrentTime < Arrival Time:

CurrentTime = Arrival Time

StartTime = CurrentTime

WaitingTime = StartTime - Arrival Time

CurrentTime = CurrentTime + Burst Time

CompletionTime = CurrentTime

TurnaroundTime = CompletionTime - Arrival Time

ResponseTime = StartTime - Arrival Time

Display Process ID

Display Waiting Time

Display Turnaround Time

Display Response Time

Calculate:

Average Waiting Time

Average Turnaround Time

Average Response Time

Display all calculated results

```

main.py Visualize Run Output
1 from collections import deque
2 # -----
3 # FCFS SCHEDULING
4 # -----
5 def fcfs(processes):
6     processes = sorted(processes, key=lambda x: x["arrival"])
7     current_time = 0
8     results = []
9     gantt = []
10    for p in processes:
11        pid = p["pid"]
12        arrival = p["arrival"]
13        burst = p["burst"]
14        if current_time < arrival:
15            current_time = arrival
16        start = current_time
17        waiting = start - arrival
18        current_time += burst
19        completion = current_time
20        turnaround = completion - arrival
21        response = start - arrival
22        results.append({
23            "pid": pid,
24            "waiting": waiting,
25            "turnaround": turnaround,
26            "response": response
27        })
28    return results
29
30 # -----
31 # UNIVERSITY WORKLOAD
32 # -----
33
34 Process Arrival Burst
35 P1 0 5
36 P2 1 8
37 P3 2 4
38 P4 3 3
39 P5 4 10
40 P6 5 7
41 P7 6 4
42 P8 7 6
43
44 -----
45 1. FCFS SCHEDULING
46 -----
47
48 GANTT CHART
49 | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 |
50 0 --> 5 --> 13 --> 17 --> 20 --> 30 --> 37 --> 41 --> 47
51
52 Process Waiting Turnaround Response

```

The output displays the execution order and the calculated waiting time, turnaround time and response time for each process.

Round Robin

Round Robin assigns each ready process a fixed time quantum. After the quantum expires, an unfinished process is moved to the end of the ready queue.

A time quantum of **3 units** is used.

PSEUDOCODE:

BEGIN

Input processes and Time Quantum

Set Remaining Time = Burst Time

Create Ready Queue

While processes are not completed:

 Add arrived processes to Ready Queue

 Select first process from Queue

 Execute for:

 Time Quantum OR Remaining Time

 Update Remaining Time and Current Time

 If process is incomplete:

 Add it to the end of Queue

 Else:

 Calculate Completion Time

 Calculate Turnaround Time

 Calculate Waiting Time

Calculate average times

Display results

END

The output is used to compare the response and waiting characteristics of Round Robin with FCFS.

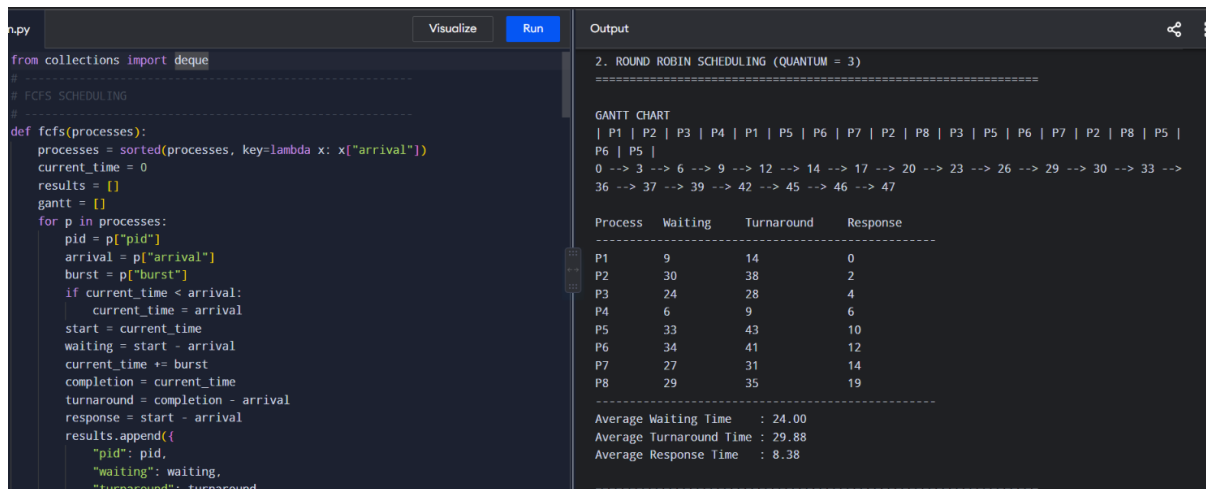
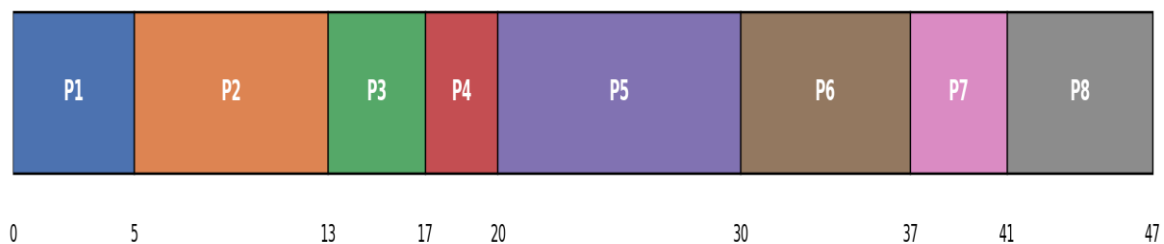
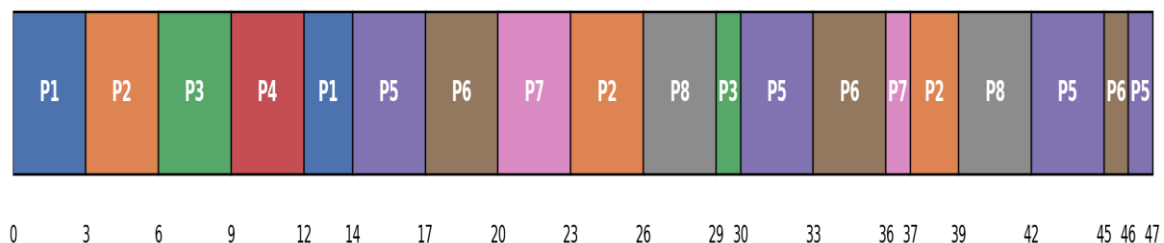


Figure: CPU Scheduling – Gantt Charts for FCFS and Round Robin (Quantum = 3)

FCFS - Gantt Chart



Round Robin (Quantum = 3) - Gantt Chart



3.4 Synchronization

The university examination database is treated as a shared resource.

Multiple readers can access the database simultaneously, but a writer requires exclusive access. The Readers–Writers problem is therefore used to demonstrate synchronization.

The implementation ensures that concurrent operations do not produce inconsistent shared data.

PSEUDOCODE:

```
BEGIN
Initialize read_count = 0
For each Reader:
    Lock read_count
    Increase read_count
    If first reader:
        Lock shared resource
    Unlock read_count
    Read shared data
    Lock read_count
    Decrease read_count
    If last reader:
        Unlock shared resource
    Unlock read_count
For Writer:
    Lock shared resource
    Write to shared data
    Unlock shared resource
Display synchronization result
END
```



```
import threading
import time
shared_data = "EXAMINATION DATABASE"
read_count = 0
read_count_lock = threading.Lock()
resource_lock = threading.Lock()
def reader(reader_id):
    global read_count
    with read_count_lock:
        read_count += 1
        if read_count == 1:
            resource_lock.acquire()
    print(f"Reader {reader_id} -> Reading shared data")
    time.sleep(1)
    print(f"Reader {reader_id} -> Finished reading")
    with read_count_lock:
        read_count -= 1
        if read_count == 0:
            resource_lock.release()
def writer(writer_id):
    print(f"Writer {writer_id} -> Waiting for exclusive access")
    with resource_lock:
        print(f"Writer {writer_id} -> Writing to shared data")
        time.sleep(1)
```

```
=====
Shared Resource:
EXAMINATION DATABASE

READERS-WRITERS SIMULATION
=====
Reader 1 -> Reading shared data
Reader 2 -> Reading shared data
Writer 1 -> Waiting for exclusive access
Reader 3 -> Reading shared data
Reader 1 -> Finished reading
Reader 2 -> Finished reading
Reader 3 -> Finished reading
Writer 1 -> Writing to shared data
Writer 1 -> Finished writing

=====
Race Condition      : PREVENTED
Shared Data Status  : CONSISTENT
Synchronization    : SUCCESSFUL
=====
Synchronization module completed.
```

The output should demonstrate readers accessing the resource, writers waiting when necessary and exclusive access being provided to the writer.

3.5 Deadlock Avoidance

University processes may require several resources simultaneously. If each process holds one resource while waiting for another, the system can potentially enter a deadlock situation.

PSEUDOCODE:

BEGIN

Input Allocation, Maximum and Available matrices

Calculate:

Need = Maximum - Allocation

Set Work = Available

While processes remain:

Find a process whose Need $\leq$ Work

If found:

Work = Work + Allocation

Mark process as completed

Add process to Safe Sequence

If no process is found:

System is UNSAFE

Stop

If all processes complete:

System is SAFE

Display Safe Sequence

END

```

n.py Visualize Run Output
def calculate_need(maximum, allocation):
    need = []
    for i in range(len(maximum)):
        row = []
        for j in range(len(maximum[i])):
            row.append(maximum[i][j] - allocation[i][j])
        need.append(row)
    return need

def is_safe(allocation, maximum, available):
    n = len(allocation)
    need = calculate_need(maximum, allocation)
    work = available.copy()
    finish = [False] * n
    safe_sequence = []
    while len(safe_sequence) < n:
        found = False
        for i in range(n):
            if not finish[i]:
                possible = True
                for j in range(len(work)):
                    if need[i][j] > work[j]:
                        possible = False
                        break
                if possible:
                    safe_sequence.append(i)
                    finish[i] = True
                    for j in range(len(work)):
                        work[j] -= need[i][j]
        if not found:
            return False
    return safe_sequence

# Example usage
maximum = [
    [2, 1, 1, 2],
    [3, 2, 1, 1],
    [3, 1, 2, 1],
    [4, 2, 1, 2]
]
allocation = [
    [0, 1, 0, 1],
    [2, 0, 0, 0],
    [3, 0, 2, 1],
    [2, 1, 1, 0]
]
available = [1, 1, 1, 1]

safe_sequence = is_safe(allocation, maximum, available)
print(safe_sequence)

```

Output

```

=====
UNICORE - BANKER'S ALGORITHM
=====

Resources:
R1 R2 R3 R4

Allocation Matrix
-----
0 1 0 1
2 0 0 0
3 0 2 1
2 1 1 0

Maximum Matrix
-----
2 1 1 2
3 2 1 1
3 1 2 1
4 2 1 2

Need Matrix
-----
2 0 1 1
1 2 1 1
0 1 0 0
2 1 0 2

```

Banker's Algorithm is implemented to determine whether the system remains in a safe state before granting resources.

The system calculates:

- Allocation matrix
- Maximum matrix
- Need matrix
- Available resources
- Safe sequence

The system also tests an unsafe resource request.

```

n.py Visualize Run Output
def calculate_need(maximum, allocation):
    need = []
    for i in range(len(maximum)):
        row = []
        for j in range(len(maximum[i])):
            row.append(maximum[i][j] - allocation[i][j])
        need.append(row)
    return need

def is_safe(allocation, maximum, available):
    n = len(allocation)
    need = calculate_need(maximum, allocation)
    work = available.copy()
    finish = [False] * n
    safe_sequence = []
    while len(safe_sequence) < n:
        found = False
        for i in range(n):
            if not finish[i]:
                possible = True
                for j in range(len(work)):
                    if need[i][j] > work[j]:
                        possible = False
                        break
                if possible:
                    safe_sequence.append(i)
                    finish[i] = True
                    for j in range(len(work)):
                        work[j] -= need[i][j]
        if not found:
            return False
    return safe_sequence

# Example usage
maximum = [
    [2, 1, 1, 2],
    [3, 2, 1, 1],
    [3, 1, 2, 1],
    [4, 2, 1, 2]
]
allocation = [
    [0, 1, 0, 1],
    [2, 0, 0, 0],
    [3, 0, 2, 1],
    [2, 1, 1, 0]
]
available = [1, 1, 1, 1]

safe_sequence = is_safe(allocation, maximum, available)
print(safe_sequence)

```

Output

```

=====
Need Matrix
-----
2 0 1 1
1 2 1 1
0 1 0 0
2 1 0 2

Available Resources:
[1, 1, 1, 1]

=====
SYSTEM STATUS : SAFE

Safe Sequence:
P2 -> P3 -> P0 -> P1

=====
UNSAFE RESOURCE REQUEST TEST
=====

Process P1 requests:
[0, 1, 1, 0]

```

The request is rejected if granting it would cause the system to enter an unsafe state.

3.6 Memory Management

During peak examination periods, many processes may simultaneously require memory. When required pages are not present in memory, page faults occur.

UniCore compares three page replacement algorithms:

- FIFO
- LRU
- Optimal

The following reference string is used:

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2

Both 3-frame and 4-frame configurations are tested.

FIFO

```
PSEUDOCODE:  
BEGIN  
Input reference string and number of frames  
Initialize empty frames and page_faults = 0  
For each page:  
    If page is not in frames:  
        If frames are not full:  
            Add page  
        Else:  
            Replace the oldest page  
    Increase page_faults  
Display page faults  
END
```

FIFO PAGE REPLACEMENT		
Page	Frames	Status
7	[7]	FAULT
0	[7, 0]	FAULT
1	[7, 0, 1]	FAULT
2	[2, 0, 1]	FAULT
0	[2, 0, 1]	HIT
3	[2, 3, 1]	FAULT
0	[2, 3, 0]	FAULT
4	[4, 3, 0]	FAULT
2	[4, 2, 0]	FAULT
3	[4, 2, 3]	FAULT
0	[0, 2, 3]	FAULT
3	[0, 2, 3]	HIT
2	[0, 2, 3]	HIT
1	[0, 1, 3]	FAULT
2	[0, 1, 2]	FAULT
Total Page Faults : 12		

LRU

PSEUDOCODE:

BEGIN

Input reference string and number of frames

Initialize empty frames and page_faults = 0

For each page:

 If page is not in frames:

 If frames are not full:

 Add page

 Else:

 Replace the least recently used page

 Increase page_faults

Display page faults

END

```
def fifo(pages, frames_count):
    frames = []
    page_faults = 0
    pointer = 0
    print("\nFIFO PAGE REPLACEMENT")
    print("-" * 60)
    print(f"{'Page':<8}{'Frames':<25}{'Status'}")
    print("-" * 60)
    for page in pages:
        if page in frames:
            status = "HIT"
        else:
            page_faults += 1
            status = "FAULT"
            if len(frames) < frames_count:
                frames.append(page)
            else:
                frames[pointer] = page
                pointer = (pointer + 1) % frames_count
        print(
            f"{'page':<8}"
            f"{'str(frames):<25}"
            f"{'status'}"
        )
    print("\nTotal Page Faults : ", page_faults)
```

LRU PAGE REPLACEMENT		
Page	Frames	Status
7	[7]	FAULT
0	[7, 0]	FAULT
1	[7, 0, 1]	FAULT
2	[2, 0, 1]	FAULT
0	[2, 0, 1]	HIT
3	[2, 0, 3]	FAULT
0	[2, 0, 3]	HIT
4	[4, 0, 3]	FAULT
2	[4, 0, 2]	FAULT
3	[4, 3, 2]	FAULT
0	[0, 3, 2]	FAULT
3	[0, 3, 2]	HIT
2	[0, 3, 2]	HIT
1	[1, 3, 2]	FAULT
2	[1, 3, 2]	HIT

Total Page Faults : 10

Optimal

PSEUDOCODE:

BEGIN

Input reference string and number of frames

Initialize empty frames and page_faults = 0

For each page:

 If page is not in frames:

 If frames are not full:

 Add page

 Else:

 Find the page used farthest in the future

 Replace that page

 Increase page_faults

Display page faults

END

FIFO — Page Faults = 12

Ref:	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2
	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0
		0	0	0	0	3	3	3	2	2	2	2	2	1	1
			1	1	1	1	0	0	0	3	3	3	3	3	2

LRU — Page Faults = 10

Ref:	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2
	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1
		0	0	0	0	0	0	0	0	3	3	3	3	3	3
			1	1	1	3	3	3	2	2	2	2	2	2	2

Optimal — Page Faults = 8

Ref:	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2
	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2
		0	0	0	0	0	0	4	4	4	0	0	0	1	1
			1	1	1	3	3	3	3	3	3	3	3	3	3

```

def fifo(pages, frames_count):
    frames = []
    page_faults = 0
    pointer = 0
    print("\nFIFO PAGE REPLACEMENT")
    print("-" * 60)
    print(f"{'Page':<8}{'Frames':<25}{'Status'}")
    print("-" * 60)
    for page in pages:
        if page in frames:
            status = "HIT"
        else:
            page_faults += 1
            status = "FAULT"
            if len(frames) < frames_count:
                frames.append(page)
            else:
                frames[pointer] = page
                pointer = (pointer + 1) % frames_count
        print(
            f"{'page':<8}"
            f"{'str(frames):<25}"
            f"{'status'}"
        )
    
```

Total Page Faults : 10

OPTIMAL PAGE REPLACEMENT

Page	Frames	Status
7	[7]	FAULT
0	[7, 0]	FAULT
1	[7, 0, 1]	FAULT
2	[2, 0, 1]	FAULT
0	[2, 0, 1]	HIT
3	[2, 0, 3]	FAULT
0	[2, 0, 3]	HIT
4	[2, 4, 3]	FAULT
2	[2, 4, 3]	HIT
3	[2, 4, 3]	HIT
0	[2, 0, 3]	FAULT
3	[2, 0, 3]	HIT
2	[2, 0, 3]	HIT
1	[2, 1, 3]	FAULT
2	[2, 1, 3]	HIT

Total Page Faults : 8

Page Replacement Comparison

Total Page Faults : 7		
=====		
PAGE REPLACEMENT COMPARISON		
=====		
Algorithm	3 Frames	4 Frames

FIFO	12	9
LRU	10	7
Optimal	8	7
=====		

The comparison is based on the total number of page faults generated by each algorithm for three and four frames.

3.7 File Allocation

University systems store different types of files such as student records, examination records, lecture videos, research data and backup files.

UniCore implements three file allocation techniques:

- Contiguous allocation
- Linked allocation
- Indexed allocation

Contiguous Allocation

In contiguous allocation, the blocks belonging to a file are stored consecutively.

PSEUDOCODE:

```
BEGIN
Input file name and required blocks
Find a contiguous sequence of free blocks

IF enough consecutive blocks are available:
    Allocate the blocks to the file
    Display allocated blocks
ELSE:
    Display "Allocation Failed"

END
```

```
def contiguous_allocation(files, disk_size):
    print("\n" + "=" * 65)
    print("CONTIGUOUS FILE ALLOCATION")
    print("=" * 65)
    disk = [None] * disk_size
    for filename, size in files:
        allocated = False
        for start in range(disk_size - size + 1):
            if all(
                disk[i] is None
                for i in range(start, start + size)
            ):
                blocks = list(
                    range(start, start + size)
                )
                for block in blocks:
                    disk[block] = filename
                print(
                    f"{{filename:<20}} "
                    f"Blocks: {{blocks}}"
                )
                allocated = True
                break
```

```
=====
UNICORE - FILE ALLOCATION MODULE
=====

University Files
-----
StudentRecords    Size = 4 blocks
ExamRecords       Size = 5 blocks
LectureVideos     Size = 8 blocks
ResearchData      Size = 6 blocks
BackupFiles       Size = 7 blocks

=====
CONTIGUOUS FILE ALLOCATION
=====
StudentRecords    Blocks: [0, 1, 2, 3]
ExamRecords       Blocks: [4, 5, 6, 7, 8]
LectureVideos     Blocks: [9, 10, 11, 12, 13, 14, 15, 16]
ResearchData      Blocks: [17, 18, 19, 20, 21, 22]
BackupFiles       Blocks: [23, 24, 25, 26, 27, 28, 29]

=====
LINKED FILE ALLOCATION
=====
```

Linked Allocation

In linked allocation, file blocks may be located at different positions and are connected through pointers.

PSEUDOCODE:

BEGIN

Input file name and required blocks

Find free blocks anywhere on the disk

IF enough free blocks are available:

 Allocate the blocks

 Link each block to the next block

 Display the block chain

ELSE:

 Display "Allocation Failed"

END

```
def linked_allocation(files, disk_size):
    print("\n" + "=" * 65)
    print("LINKED FILE ALLOCATION")
    print("=" * 65)

    free_blocks = list(range(disk_size))

    for filename, size in files:
        if len(free_blocks) < size:
            print(
                f"{{filename:<20}} Allocation Failed"
            )
            continue

        blocks = free_blocks[:size]
        free_blocks = free_blocks[size:]
```

```
=====
StudentRecords    Blocks: [0, 1, 2, 3]
ExamRecords       Blocks: [4, 5, 6, 7, 8]
LectureVideos     Blocks: [9, 10, 11, 12, 13, 14, 15, 16]
ResearchData      Blocks: [17, 18, 19, 20, 21, 22]
BackupFiles       Blocks: [23, 24, 25, 26, 27, 28, 29]

=====
LINKED FILE ALLOCATION
=====
StudentRecords    Chain: 0 -> 1 -> 2 -> 3
ExamRecords       Chain: 4 -> 5 -> 6 -> 7 -> 8
LectureVideos     Chain: 9 -> 10 -> 11 -> 12 -> 13 -> 14 -> 15 -> 16
ResearchData      Chain: 17 -> 18 -> 19 -> 20 -> 21 -> 22
BackupFiles       Chain: 23 -> 24 -> 25 -> 26 -> 27 -> 28 -> 29

=====
INDEXED FILE ALLOCATION
=====
File : StudentRecords
```


Indexed Allocation

Indexed allocation uses an index block containing addresses of the data blocks belonging to a file.

PSEUDOCODE:

BEGIN

Input file name and required blocks

Allocate one index block

Find required free data blocks

IF enough blocks are available:

 Store data-block addresses in the index block

 Display index block and data blocks

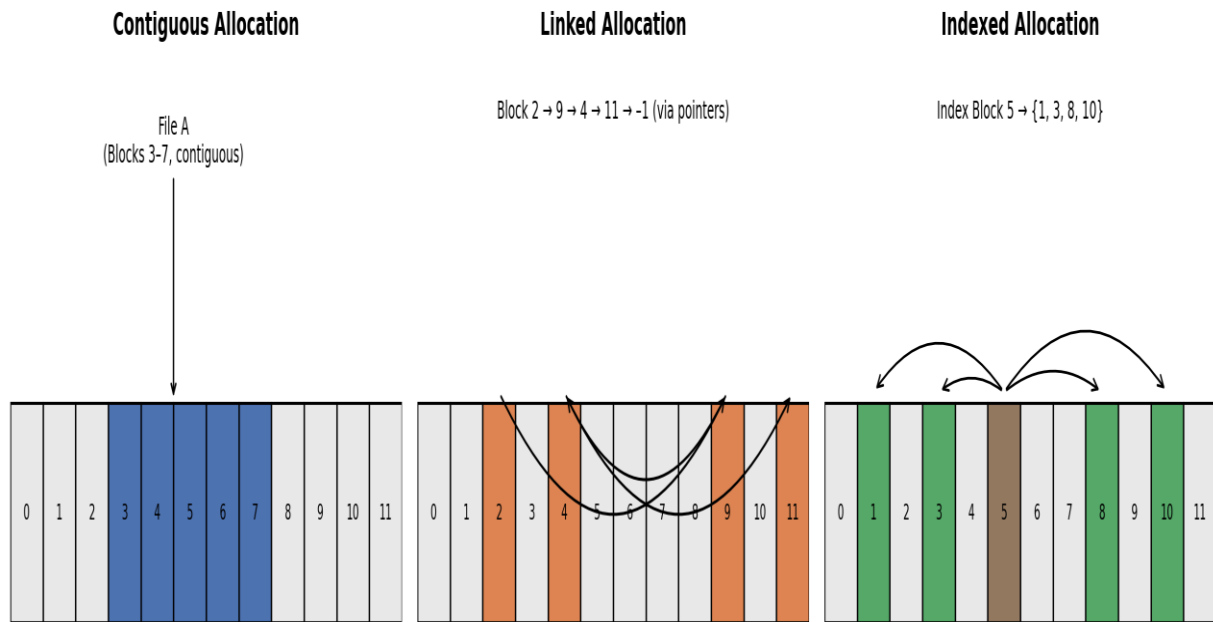
ELSE:

 Display "Allocation Failed"

END

```
def indexed_allocation(files, disk_size):  
  
    print("\n" + "=" * 65)  
    print("INDEXED FILE ALLOCATION")  
    print("=" * 65)  
  
    free_blocks = list(range(disk_size))  
  
    for filename, size in files:  
        if len(free_blocks) < size + 1:  
            print(  
                f"{filename:<20} Allocation Failed"  
            )  
            continue  
  
        index_block = free_blocks.pop(0)  
  
        data_blocks = []  
  
        =====  
        INDEXED FILE ALLOCATION  
        =====  
  
        File : StudentRecords  
        Index Block : 0  
        Data Blocks : [1, 2, 3, 4]  
  
        File : ExamRecords  
        Index Block : 5  
        Data Blocks : [6, 7, 8, 9, 10]  
  
        File : LectureVideos  
        Index Block : 11  
        Data Blocks : [12, 13, 14, 15, 16, 17, 18, 19]  
  
        File : ResearchData  
        Index Block : 20  
        Data Blocks : [21, 22, 23, 24, 25, 26]  
  
        File : BackupFiles  
        Index Block : 27  
        Data Blocks : [28, 29, 30, 31, 32, 33, 34]
```

Figure: Conceptual Block Diagrams – Contiguous, Linked and Indexed File Allocation



3.8 Disk Scheduling

University services generate multiple disk I/O requests. Efficient disk scheduling can reduce unnecessary movement of the disk head.

The simulated disk contains **200 cylinders numbered from 0 to 199**.

Initial head position:

53

Request sequence:

98 183 37 122 14 124 65 67 45 150

The following algorithms are implemented:

- FCFS
- SSTF
- SCAN

FCFS Disk Scheduling

PSEUDOCODE:

```

BEGIN
Input disk requests and initial head position
Set total movement = 0
For each request in given order:
    Calculate distance from current head
    Add distance to total movement
    Move head to the request
Display service sequence
  
```

```
def fcfs(requests, head):
    sequence = []
    movement = 0
    current = head
    for request in requests:
        movement += abs(current - request)
        current = request
        sequence.append(request)
    return sequence, movement

def sstf(requests, head):
    remaining = requests.copy()
    sequence = []
    movement = 0
    current = head
    while remaining:
        closest = min(
            remaining,
            key=lambda x: abs(x - current)
        )
        movement += abs(current - closest)
        current = closest
        sequence.append(closest)
        remaining.remove(closest)
    return sequence, movement
```

```
=====
UNICORE - DISK SCHEDULING MODULE
=====

Disk Cylinders : 0 - 199
Initial Head   : 53

Requests:
[98, 183, 37, 122, 14, 124, 65, 67, 45, 150]

-----
1. FCFS
-----

Service Sequence:
98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67 -> 45 -> 150

Total Head Movement : 767
-----
```

SSTF Disk Scheduling

PSEUDOCODE:

BEGIN

Input disk requests and initial head position

Set total movement = 0

WHILE requests remain:

 Find the request closest to current head

 Move head to that request

 Add movement to total

 Mark request as completed

Display service sequence

Display total head movement

END

```
def sstf(requests, head):
    remaining = requests.copy()
    sequence = []
    movement = 0
    current = head
    while remaining:
        closest = min(
            remaining,
            key=lambda x: abs(x - current)
        )
        movement += abs(current - closest)
        current = closest
        sequence.append(closest)
        remaining.remove(closest)
    return sequence, movement

def scan(requests, head, disk_size=200):
    left = sorted([r for r in requests if r < head])
    right = sorted([r for r in requests if r >= head])
    sequence = []
    movement = 0
    current = head

    # Move right first
    for request in right:
        movement += abs(current - request)
        sequence.append(request)
        current = request

    # Now move left
    for request in reversed(left):
        movement += abs(current - request)
        sequence.append(request)
        current = request

    return sequence, movement
```

```
-----
2. SSTF
-----
Service Sequence:
45 -> 37 -> 14 -> 65 -> 67 -> 98 -> 122 -> 124 -> 150 -> 183

Total Head Movement : 208

-----
3. SCAN
-----
Service Sequence:
65 -> 67 -> 98 -> 122 -> 124 -> 150 -> 183 -> 45 -> 37 -> 14

Total Head Movement : 331

=====
DISK SCHEDULING COMPARISON
=====
```

SCAN Disk Scheduling

PSEUDOCODE:

BEGIN

Input disk requests and initial head position

Separate requests into left and right of head

Sort both groups

Move head in the selected direction

Service requests along the way

Continue to disk end

Reverse direction

Service remaining requests

Calculate total head movement

Display service sequence

Display total head movement

END

```
def sstf(requests, head):
    remaining = requests.copy()
    sequence = []
    movement = 0
    current = head
    while remaining:
        closest = min(
            remaining,
            key=lambda x: abs(x - current)
        )
        movement += abs(current - closest)
        current = closest
        sequence.append(closest)
        remaining.remove(closest)
    return sequence, movement

def scan(requests, head, disk_size=200):
    left = sorted([r for r in requests if r < head])
    right = sorted([r for r in requests if r >= head])
    sequence = []
    movement = 0
    current = head

    # Move right first
    for request in right:
        movement += abs(current - request)
        sequence.append(request)
        current = request

    # Now move left
    for request in reversed(left):
        movement += abs(current - request)
        sequence.append(request)
        current = request

    return sequence, movement
```

```
-----
2. SSTF
-----
Service Sequence:
45 -> 37 -> 14 -> 65 -> 67 -> 98 -> 122 -> 124 -> 150 -> 183

Total Head Movement : 208

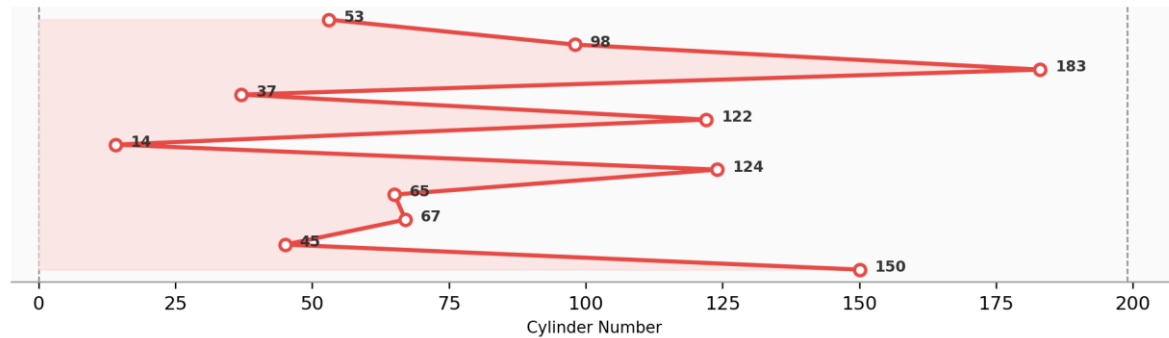
-----
3. SCAN
-----
Service Sequence:
65 -> 67 -> 98 -> 122 -> 124 -> 150 -> 183 -> 45 -> 37 -> 14

Total Head Movement : 331

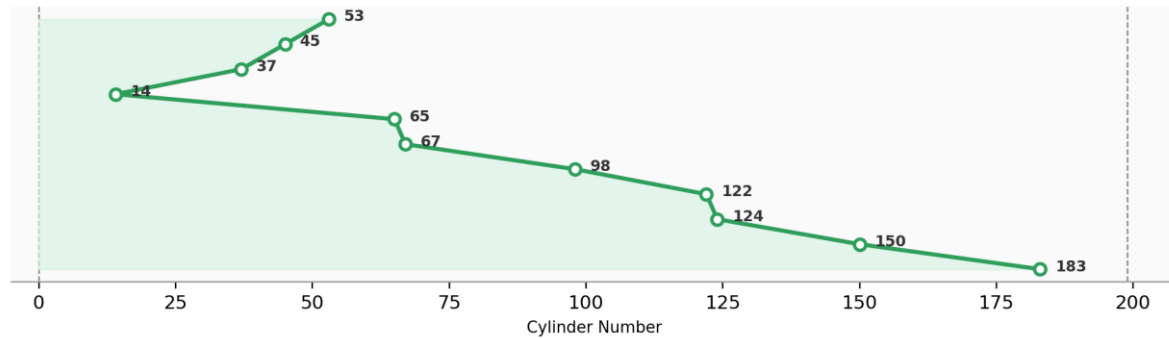
=====
DISK SCHEDULING COMPARISON
=====
Algorithm      Head Movement
-----
FCFS           767
SSTF           208
SCAN           331
=====
```

Figure: Disk Head Movement Path – FCFS, SSTF and SCAN Scheduling

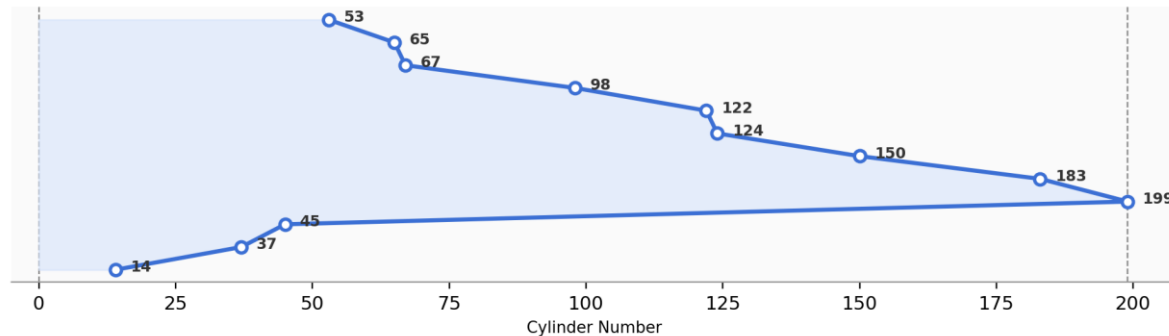
FCFS (Total Head Movement = 767)



SSTF (Total Head Movement = 208)



SCAN (Total Head Movement = 331)



4. COURSE KNOWLEDGE

The UniCore project applies several fundamental Operating System concepts.

Process Management

CPU scheduling determines the order in which processes receive processor time. FCFS is simple and follows arrival order, while Round Robin provides time-sharing and improved fairness.

Process Synchronization

Synchronization mechanisms control access to shared resources. The Readers–Writers problem demonstrates how multiple readers and exclusive writers can safely operate on shared examination data.

Deadlock Management

Banker's Algorithm checks whether resource allocation keeps the system in a safe state and prevents unsafe allocations.

Memory Management

Paging divides memory into fixed-size pages and frames. Page replacement algorithms determine which page should be removed when memory is full.

File Management

File allocation determines how data blocks are organized on storage. Contiguous, linked and indexed allocation provide different advantages depending on access patterns.

Disk Management

Disk scheduling determines the order in which I/O requests are serviced. FCFS emphasizes simplicity, SSTF reduces immediate movement and SCAN provides a more balanced movement pattern.

5. IMPLEMENTATION / SOURCE CODE

The UniCore implementation is divided into separate programs so that each OS concept can be tested independently.

UniCore/

```
|  
|— cpu_scheduling.py  
|— readers_writers.py  
|— bankers_algorithm.py  
|— page_replacement.py  
|— file_allocation.py  
|— disk_scheduling.py  
|— unicoresystem.py
```

Each individual program produces its own output before being integrated into the final UniCore system.

6. TEST CASES AND RESULTS

Test Case 1 — Normal Workload

A normal university workload is simulated with approximately 320 active users.

```
def main():
    ]

    print("\nUNIVERSITY WORKLOAD")
    print("-" * 50)

    print(f"{'Process':<10}{ 'Arrival':<12}{ 'Burst':<10}")

    for p in processes:
        print(f"{'pid':<10}{p['arrival']:<12}{p['burst']:<10}")

    # FCFS
    fcfs_results, fcfs_gantt = fcfs(processes)
    display_results("1. FCFS SCHEDULING", fcfs_results, fcfs_gantt)

    # Round Robin
    rr_results, rr_gantt = round_robin(processes, 3)
    display_results("2. ROUND ROBIN SCHEDULING (QUANTUM = 3)",
        rr_results, rr_gantt)

    print("\n" + "=" * 65)
    print("CPU SCHEDULING MODULE COMPLETED")
    print("=" * 65)
```

```
=====
UNICORE - CPU SCHEDULING MODULE
=====

UNIVERSITY WORKLOAD
=====
Process  Arrival  Burst
P1       0        5
P2       1        8
P3       2        4
P4       3        3
P5       4       10
P6       5        7
P7       6        4
P8       7        6
=====

1. FCFS SCHEDULING
=====

GANTT CHART
| P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 |
0 --> 5 --> 13 --> 17 --> 20 --> 30 --> 37 --> 41 --> 47
```

The purpose of this test is to verify that the individual OS modules operate correctly under moderate resource demand.

Test Case 2 — Peak Examination Workload

The second test represents an examination period with approximately 800 concurrent users.

```
from collections import deque

# =====
# CPU SCHEDULING
# =====

def fcfs(processes):
    current_time = 0
    waiting = []
    turnaround = []
    response = []
    for p in processes:
        if current_time < p["arrival"]:
            current_time = p["arrival"]
        start = current_time
        waiting.append(
            start - p["arrival"]
        )
        response.append(
            start - p["arrival"]
        )
        current_time += p["burst"]
        turnaround.append(
            current_time - p["arrival"]
        )
    return (
```

```
=====
UNICORE
INTELLIGENT OS RESOURCE ORCHESTRATION
=====

SCENARIO
=====
Smart University - Peak Examination Workload
Concurrent Users : 800
CPU Processes    : 8
Physical RAM     : 16 GB
Disk Cylinders   : 0 - 199
=====

1. CPU SCHEDULING
=====

Metric           FCFS      Round Robin
=====
Average Waiting Time  16.88    24.00
Average Turnaround Time 22.75    29.88
Average Response Time  16.88     8.38
=====
```

This test evaluates the behaviour of the system under increased CPU, memory and I/O pressure.

Test Case 3 — Unsafe Resource Request

A resource request is generated that would result in an unsafe state.

```
def main():
    if safe:
        print(" -> ".join(
            f"P{i}" for i in sequence
        ))
    else:
        print("SYSTEM STATUS : UNSAFE")

    print("-" * 65)

    # Test a resource request
    request = [0, 1, 1, 0]

    test_request(
        allocation,
        maximum,
        available,
        1,
        request
    )
```

Resources:
R1 R2 R3 R4
Allocation Matrix

0 1 0 1
2 0 0 0
3 0 2 1
2 1 1 0
Maximum Matrix

2 1 1 2
3 2 1 1
3 1 2 1
4 2 1 2
Need Matrix

2 0 1 1
1 2 1 1
0 1 0 0
2 1 0 2

The expected behaviour is that Banker's Algorithm rejects the request and maintains the system in a safe state.

Test Case 4 — Memory Pressure

The page replacement algorithms are tested using a smaller number of frames.

```
def main():

    print("\n" + "-" * 65)
    print("PAGE REPLACEMENT COMPARISON")
    print("-" * 65)

    print(
        f"{'Algorithm':<15}"
        f"{'3 Frames':<15}"
        f"{'4 Frames':<15}"
    )

    print("-" * 45)

    for algorithm in ["FIFO", "LRU", "Optimal"]:

        print(
            f"{algorithm:<15}"
            f"{results[3][algorithm]:<15}"
            f"{results[4][algorithm]:<15}"
        )
```

Page	Frames	Status
7	[7]	FAULT
0	[7, 0]	FAULT
1	[7, 0, 1]	FAULT
2	[7, 0, 1, 2]	FAULT
0	[7, 0, 1, 2]	HIT
3	[3, 0, 1, 2]	FAULT
0	[3, 0, 1, 2]	HIT
4	[3, 0, 4, 2]	FAULT
2	[3, 0, 4, 2]	HIT
3	[3, 0, 4, 2]	HIT
0	[3, 0, 4, 2]	HIT
3	[3, 0, 4, 2]	HIT
2	[3, 0, 4, 2]	HIT
1	[1, 0, 4, 2]	FAULT
2	[1, 0, 4, 2]	HIT

Total Page Faults : 7

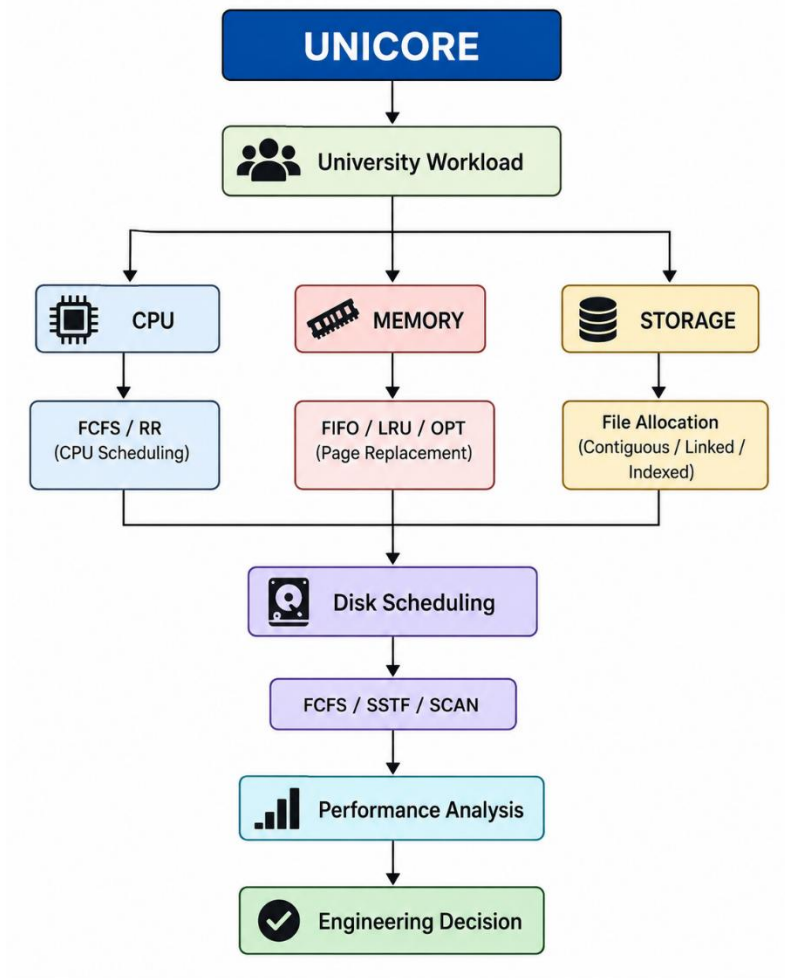
=====

PAGE REPLACEMENT COMPARISON

The resulting page-fault counts are compared to identify the effect of limited memory availability.

7. ARCHITECTURE / DESIGN DIAGRAM

The UniCore architecture connects the university workload to the different OS management modules.



8. COMPARATIVE ANALYSIS

The results obtained from the individual modules are combined to compare the algorithms.

CPU Scheduling

Algorithm	Avg. Waiting Time	Avg. Turnaround Time	Avg. Response Time
FCFS	16.88	22.75	16.88
Round Robin	24.00	29.88	8.38

Memory Management

Algorithm	3 Frames	4 Frames
FIFO	12	9
LRU	10	7
Optimal	8	7

Disk Scheduling

Algorithm	Total Head Movement
FCFS	767
SSTF	208
SCAN	331

9. FINAL ENGINEERING DECISION

Based on the experimental results, UniCore selects suitable OS strategies for the smart university workload.

The CPU scheduling decision considers waiting time, turnaround time, response time and fairness.

The memory decision considers page-fault frequency under different frame configurations.

The disk decision considers total head movement together with fairness and starvation risk.

For shared resources, synchronization is selected to maintain consistent access. Banker's Algorithm is used for deadlock avoidance because it prevents resource requests that would place the system in an unsafe state.

The final decisions will be filled using the **actual numerical results obtained from the implemented programs**

OS Component	Selected Strategy
CPU Scheduling	Round Robin
Synchronization	Readers–Writers
Deadlock Avoidance	Banker's Algorithm

Page Replacement	LRU
File Allocation	Indexed Allocation
Disk Scheduling	SCAN

10. BROADER CONSIDERATIONS

Reliability and Safety

UniCore considers reliability by preventing unsafe resource allocation and protecting shared examination data through synchronization.

Sustainability

Efficient CPU, memory and disk management can reduce unnecessary resource consumption and improve overall system utilization.

Accessibility and Society

During online examinations, fair resource allocation is important because many students depend on the system simultaneously. Scheduling strategies should therefore avoid unnecessary delays for interactive services.

Ethics and Professional Responsibility

University systems may handle student records and examination information. Therefore, responsible resource management, controlled access and protection of shared data are important considerations.

11. CONCLUSION

UniCore demonstrates the integration of fundamental Operating System concepts within a realistic smart university environment.

The system implements CPU scheduling, synchronization, deadlock avoidance, memory management, file allocation and disk scheduling as separate modules and then combines their results for overall analysis.

FCFS and Round Robin are evaluated for CPU scheduling, Readers–Writers is used for synchronization, Banker’s Algorithm handles deadlock avoidance, FIFO/LRU/Optimal are

evaluated for page replacement, Contiguous/Linked/Indexed are compared for file allocation, and FCFS/SSTF/SCAN are evaluated for disk scheduling.

The project demonstrates that algorithm selection should depend on workload characteristics rather than assuming that one algorithm is universally optimal.

The final UniCore design therefore uses experimental results to support its engineering decisions and demonstrates how Operating System concepts can work together to provide fair, reliable and efficient resource management.

12. STUDENT REFLECTION

Through this project, I gained a better understanding of how different Operating System components interact with each other.

Initially, CPU scheduling, memory management, deadlock handling and disk scheduling appeared to be separate concepts. However, implementing them within a common university scenario showed that they are connected through the shared goal of efficient resource management.

The most important learning was that an algorithm should not be selected only because it is theoretically efficient. Its suitability depends on the workload, performance requirements and system constraints.

If the number of concurrent university users increases significantly, I would improve UniCore by introducing adaptive scheduling, dynamic memory management and workload-aware resource allocation.

Overall, the project helped me understand how Operating System concepts can be applied to a realistic computing environment rather than being treated only as individual algorithms.

13. REFERENCES

1. Celis, J.R., Gonzales, D., Lagda, E. and Rutaquio, L. Jr. (2014) '*A comprehensive review for disk scheduling algorithms*', *International Journal of Computer Science Issues*, 11(1), pp.74–79.
2. Courtois, P.J., Heymans, F. and Parnas, D.L. (1971) '*Concurrent control with “readers” and “writers”*', *Communications of the ACM*, 14(10), pp.667–668.
3. Rahman, S.H.U. and Siddiqua, A. (2021) '*Hybrid indexed and linked list file allocation system*', *International Research Journal of Modernization in Engineering Technology and Science*, 3(6), pp.518–521.
4. Rexha, G., Elmazi, E. and Tafa, I. (2015) '*A comparison of three page replacement algorithms: FIFO, LRU and Optimal*', *Academic Journal of Interdisciplinary Studies*, 4(2 S2), p.56.
5. Siahaan, A.P.U. (2016) '*Comparison analysis of CPU scheduling: FCFS, SJF and Round Robin*', *International Journal of Engineering and Development Research*, 4(3), pp.124–131.
6. Zohora, M.F., Farhin, F. and Kaiser, M.S. (2024) '*DBDAA: a real-time approach to Dynamic Banker’s Deadlock Avoidance Algorithm with optimized time complexity*', *PLOS ONE*, 19(9), e0310807.